

Capstone Project: End-to-End Retail Data Pipeline

Architecture: Medallion (RAW → SILVER → GOLD)

Tech Stack: Databricks (Spark), Snowflake, Airflow, Python, SQL

1. Project Objective

Build a production-grade batch data pipeline that ingests raw retail CSV data, applies rigorous data engineering transformations using the Medallion Architecture, ensures high data quality, and synchronizes the final analytical models into Snowflake for business intelligence.

2. High-Level Architecture

graph LR

subgraph "Source (CSV)"

S1[(Customers)]

S2[(Products)]

S3[(Orders)]

S4[(Items)]

end

subgraph "Databricks (Spark & Delta Lake)"

B[Bronze: Raw Ingestion] --> S[Silver: Cleaned & Validated]

S --> G[Gold: Business Modeling]

end

subgraph "Warehouse"

G --> SNOW[(Snowflake)]

end

subgraph "Orchestration"

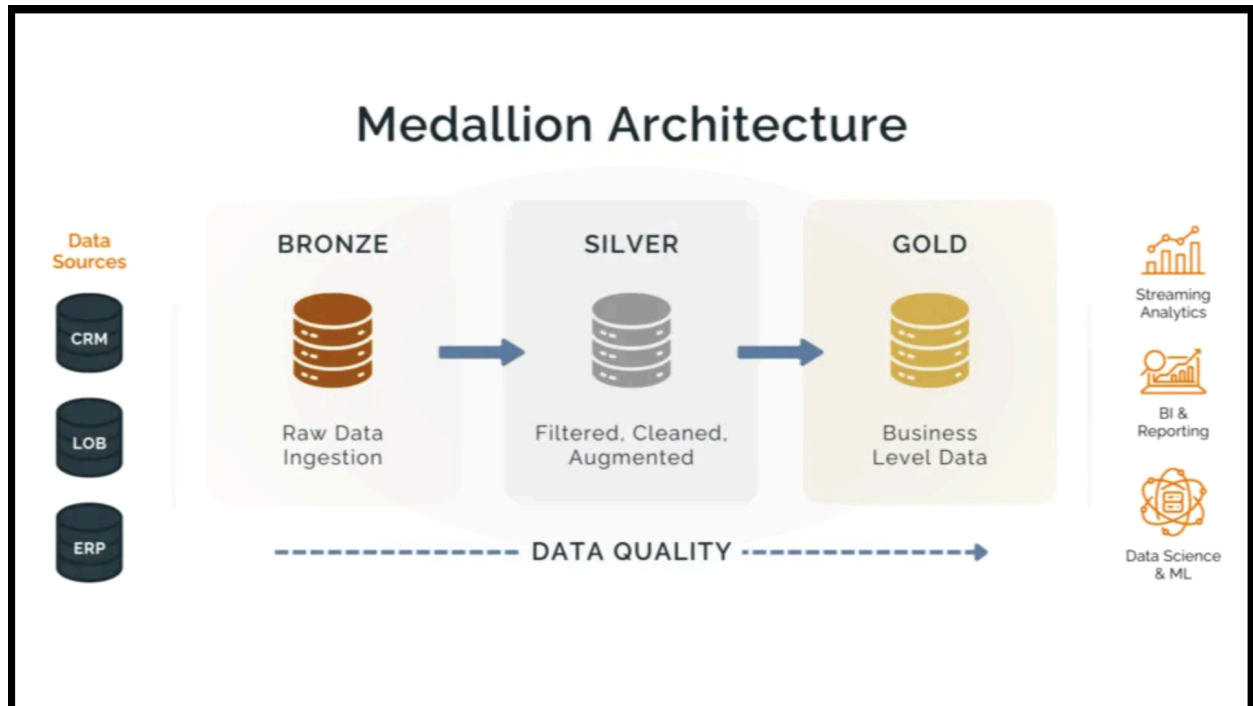
A[Apache Airflow] -.-> B

A -.-> S

A -.-> G

A -.-> SNOW

End



3. Mandatory Datasets

You must use the following schema for your input files.

Dataset	Primary Key	Key Columns
customers.csv	customer_id	first_name, last_name, email, signup_date, country
products.csv	product_id	product_name, category, price (double)
orders.csv	order_id	customer_id, order_date, status (CREATED/SHIPPED/DELIVERED/CANCELLED)
order_items.csv	order_item_id	order_id, product_id, quantity (int)

4. Project Structure (Mandatory)

Adhere to this folder structure to maintain clean code principles:

```
capstone-project/
├── data/raw/          # Input CSV files
├── src/               # PySpark Processing Logic
│   ├── ingestion.py   # Bronze layer logic
│   ├── silver_transform.py # Cleaning & Validation logic
│   ├── gold_transform.py # Dimensional modeling
│   └── data_quality.py # Custom DQ framework
├── sql/
│   └── gold_tables.sql # DDL for Snowflake
├── airflow/
│   └── retail_dag.py   # DAG Definition
├── config/
│   └── config.yaml     # Environment & Table configs
└── README.md          # Documentation & Setup Guide
```

5. Implementation Phases

Phase 1: Bronze (RAW) - Ingestion

Task: Read CSVs and persist as Delta Tables.

- **Rule 1:** Zero transformations (Keep data in original state).
- **Rule 2:** Append Metadata: ingestion_timestamp and source_file_name.
- **Output:** bronze_customers, bronze_products, etc.

Phase 2: Silver - Cleaning & Validation

Task: Ensure the data is "Trustworthy."

- **Customer Validations:** Remove duplicate customer_id; Validate email format (must contain @).
- **Product Validations:** price must be > 0.
- **Order Validations:** Filter for specific statuses; order_date cannot be NULL.
- **Integrity:** order_id in order_items must exist in the orders table.

Phase 3: Gold - Dimensional Modeling

Task: Build a Star Schema optimized for analytics.

- **fact_sales:** Join Orders + Items + Products. Calculate total_amount = quantity * price.
- **dim_customer:** Flatten names (first + last) and include geography.

- **dim_product:** Standardized product catalog.
- **Filter:** Include ONLY DELIVERED orders in the fact table.

Phase 4: Data Quality (DQ) Checks

Required Framework:

1. **Uniqueness:** Check for PK violations in Gold tables.
2. **Completeness:** Verify no NULLs in critical columns (Date, ID, Amount).
3. **Reconciliation:** Record count check (Bronze vs. Silver).

Phase 5: Snowflake Loading

Task: Sync Gold Delta tables to Snowflake.

- Use the **Spark-Snowflake Connector**.
- **Idempotency:** Use OVERWRITE mode or MERGE logic to prevent duplicates on retries.

6. Orchestration & Performance

Airflow DAG Flow

ingest_raw → build_silver → dq_check_silver → build_gold → dq_check_gold → load_snowflake

Performance Requirements

- **Partitioning:** Partition Silver/Gold tables by order_date.
- **Optimization:** Use Broadcast Joins when joining the dim_product (small table) to fact_sales.
- **Caching:** Cache the Silver orders table if used multiple times in Gold transformations.

7. Evaluation & Acceptance Criteria

Criteria	Weight	Warning / Failure Triggers
Spark Logic	25%	Failed if using loops instead of vectorized operations.
Data Modeling	15%	Failed if Gold layer is not a proper Star Schema.
Data Quality	15%	Failed if DQ checks are not logged or automated.
Airflow DAG	15%	Failed if tasks are not modular or lack retries.

Snowflake Load	10%	Failed if data is truncated or duplicated.
Git Practice	10%	Failed if only one single "Initial Commit" exists.

8. Final Deliverables

1. **GitHub Link:** Public repository with documented code.
2. **README:** Setup instructions for Databricks secrets and Snowflake credentials.
3. **Evidence:** A PDF or Folder containing:
 - Screenshot of the successful Airflow DAG Tree View.
 - Screenshot of fact_sales query result in Snowflake.
 - Spark UI screenshot showing a Broadcast Join execution.

Prepared for: Data Engineering Freshers

Subject: Capstone Project V1.0

Status: Execution Ready